

The solution: Planning with BuiltInPlanner

Enabling multi-step reasoning

Reference: [ADK docs—planning](#)

According to ADK documentation, use [BuiltInPlanner](#) to leverage Gemini's thinking feature:

“[planner](#) (optional): Assign a [BasePlanner](#) instance to enable multi-step reasoning and planning before execution. [BuiltInPlanner](#): Leverages the model's built-in planning capabilities (e.g., Gemini's thinking feature).”

Python

```
from google.adk.agents import LlmAgent
from google.adk.planners import BuiltInPlanner
from google.genai import types

planning_agent = LlmAgent(
    model="gemini-2.5-flash",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(
            include_thoughts=True,    # Show reasoning in response
            thinking_budget=1024      # Tokens allocated for thinking
        )
    ),
    instruction="Solve complex problems systematically"
)
```

Note: This feature uses [Gemini thinking](#) capabilities.



Seeing planning in action

Let's compare the same problem with and without planning to understand what planning actually does:

Example prompt:

"Should I buy or lease a car for my 50-mile daily commute?"

Without planning

```
Python
basic_agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="Help users make decisions"
)
```

Typical response without planning:

```
None
Leasing offers lower monthly payments
and you can get a new car every few
years.
Buying means you own the car and can
drive unlimited miles. For a 50-mile
daily
commute, buying is probably better
since leases have mileage limits.
```

- ✗ No breakdown of financial implications
- ✗ No consideration of total cost over time
- ✗ Doesn't quantify the mileage issue (50 miles/day = ~18,000 miles/year)
- ✗ No comparison of different scenarios

With BuiltInPlanner

```
Python
planning_agent = LlmAgent(
    model="gemini-2.5-flash",
    planner=BuiltInPlanner(
        thinking_config=types.Thinking
        Config(
            include_thoughts=True,
            # See the reasoning process
            thinking_budget=1024
            # Allocate tokens for thinking
        )
    ),
    instruction="Help users make decisions"
)
```


Response with planning (include_thoughts=True shows the reasoning):

None

[Internal Thinking - visible because include_thoughts=True]

Let me analyze this systematically:

- 50 miles/day × 5 days/week × 52 weeks = ~13,000 miles/year
- Most leases limit 10,000-12,000 miles/year - this is borderline
- Need to compare: total cost, flexibility, mileage overage fees
- Consider: how long will they keep the car?

Key factors:

1. Lease mileage limits: 12k standard, would need 15k package (higher payment)
2. Buying: higher monthly payment but builds equity
3. High-mileage driving impacts resale value either way

Let me create scenarios...

[Final Answer]

For your 50-mile daily commute (~13,000 miles/year), here's the comparison:

****Leasing:****

- Monthly cost: \$350 (with 15,000-mile/year package - you'd exceed 12k standard)
- 3-year total: \$12,600 in payments
- End result: Return the car, no equity
- Risk: Overage fees if you exceed mileage (\$0.25/mile can add up fast)

****Buying:****

- Monthly cost: \$450 (typical 5-year loan)
- 3-year total: \$16,200 in payments
- End result: Own a car worth ~\$15,000 (accounting for high-mileage depreciation)
- Net cost: ~\$1,200 after equity

****Recommendation for your situation:****

Buy rather than lease. Your high mileage (13k/year) makes leasing risky and expensive:

1. You'd need a higher-mileage lease package (more expensive)
2. Any unexpected trips push you into costly overage fees
3. Over 3 years, buying costs only ~\$1,200 more after accounting for equity
4. Buying gives you flexibility - no mileage restrictions

Exception: If you prioritize lower monthly payment over long-term cost, a 15k-mile lease could work, but budget for potential overage fees.

What planning added:

- ✓ Problem decomposition: Broke down mileage calculation, cost factors
- ✓ Multi-factor analysis: Compared total cost, equity, risks, flexibility
- ✓ Quantified thinking: Calculated actual numbers instead of generalizations
- ✓ Scenario comparison: Examined both options with specific context
- ✓ Nuanced recommendation: Main advice + exception case

Key insight about instructions:

Notice that the instruction stayed simple: “Help users make decisions.” You don’t need to write complex instructions like “break down the problem into phases” or “consider trade-offs and constraints.” The planning capability enables this structured reasoning automatically—[thinking_budget](#) determines how deeply the model can think, and [include_thoughts](#) lets you see (and debug) that reasoning process.

Core concepts

1. Understanding [planner](#) parameter

Reference: [ADK Docs—planner parameter](#)

The [planner](#) parameter (singular, not plural) enables planning:

```
Python
from google.adk.planners import BuiltInPlanner

agent = LlmAgent(
    model="gemini-2.5-flash",
    planner=BuiltInPlanner(...) # Singular: planner=, not planners=
)
```

Key points:

- Currently Python-only feature
- Uses model’s built-in thinking capabilities
- Best for complex, multi-step problems



2. Configuring ThinkingConfig

Reference: [ADK Docs –ThinkingConfig parameters](#)

From the documentation:

“Here, the `thinking_budget` parameter guides the model on the number of thinking tokens to use when generating a response. The `include_thoughts` parameter controls whether the model should include its raw thoughts and internal reasoning process in the response.”

Python

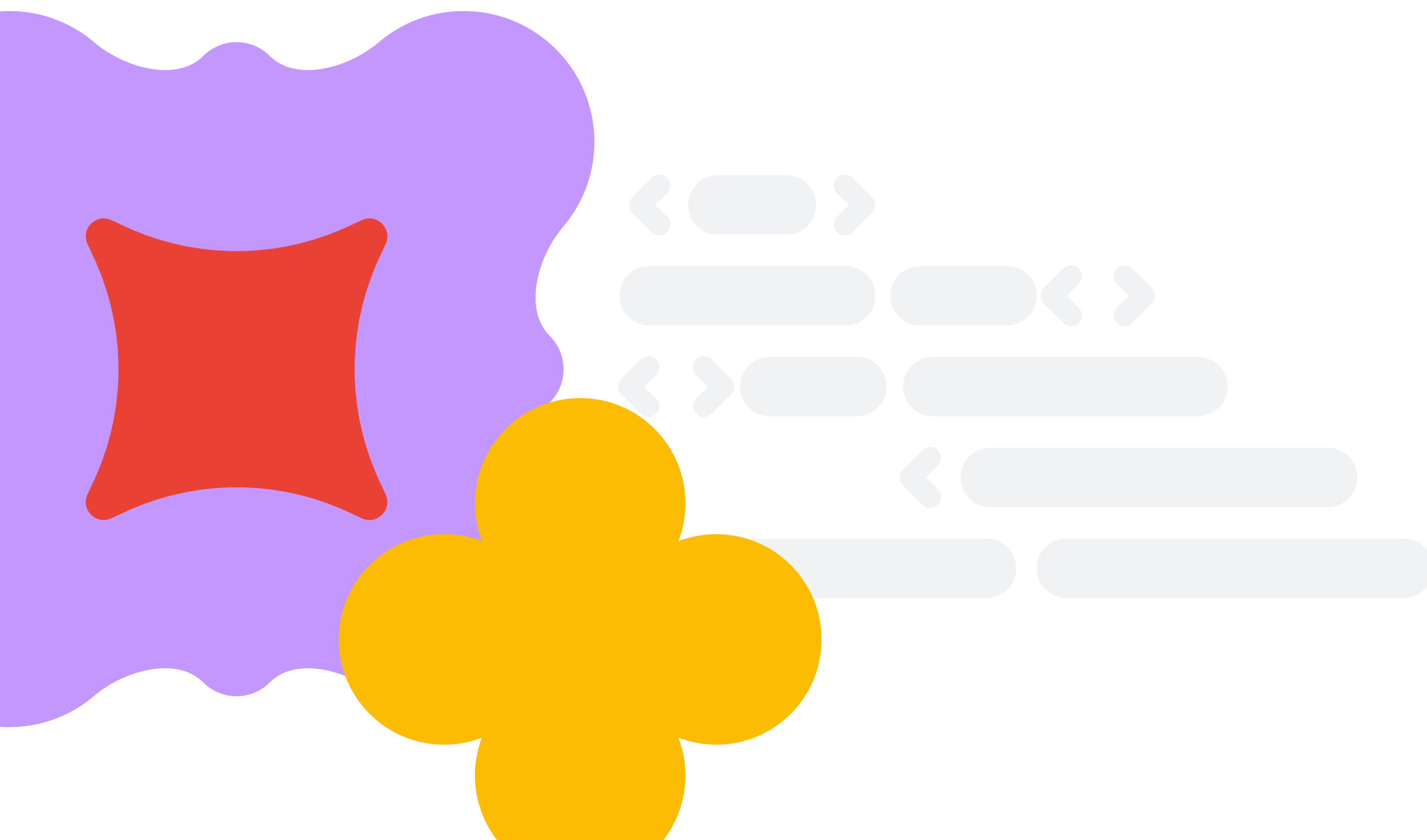
```
from google.genai import types

# Configuration with visible reasoning
transparent_config = types.ThinkingConfig(
    include_thoughts=True,    # Show the agent's reasoning
    thinking_budget=1024     # Allocate 1024 tokens for thinking
)

# Configuration with hidden reasoning
opaque_config = types.ThinkingConfig(
    include_thoughts=False,  # Hide internal reasoning
    thinking_budget=512      # Use fewer thinking tokens
)
```

Parameters explained:

- `include_thoughts`: If true, the response includes the agent's internal reasoning process
- `thinking_budget`: Number of tokens the model can use for thinking (not included in final response)





3. Choosing between planners

Reference: [ADK Docs—planner types](#)

ADK provides two planner types, each suited for different scenarios:

BuiltInPlanner (recommended for Gemini models)

Leverages the model’s native thinking capabilities. From the ADK documentation:

“**BuiltInPlanner**: Leverages the model’s built-in planning capabilities (e.g., Gemini’s thinking feature).”

Best for:

- **Gemini models:** 2.5 Flash, 2.5 Pro, etc. with built-in thinking support
- **Transparent reasoning:** See internal thought process with `include_thoughts=True`
- **Flexible control:** Adjust reasoning depth with `thinking_budget`

Configuration:

```
Python
from google.adk.planners import BuiltInPlanner
from google.genai import types

agent = LlmAgent(
    model="gemini-2.5-flash",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(
            include_thoughts=True,    # Show reasoning (great for debugging)
            thinking_budget=1024      # Tokens for thinking (512-2048 typical)
        )
    )
)
```

Key parameters:

- **thinking_budget**: Controls how deeply the model can reason. Higher values (e.g., 2048) allow more thorough analysis for complex problems. Lower values (e.g., 512) are faster for simpler tasks.
- **include_thoughts**: When True, you see the model's internal reasoning process in the response. Essential for debugging and understanding how the agent arrived at its answer.

PlanReActPlanner (For non-Gemini models)

Reference: [ADK Docs—PlanReActPlanner](#)

From the ADK documentation:

“**PlanReActPlanner**: This planner instructs the model to follow a specific structure in its output: first create a plan, then execute actions (like calling tools), and provide reasoning for its steps. *It’s particularly useful for models that don’t have a built-in “thinking” feature.*”

Best for:

- **Non-Gemini models:** e.g., other LLMs without native thinking capabilities
- **Structured output format:** Enforces PLANNING → ACTION → REASONING → FINAL_ANSWER structure
- **Tool-heavy workflows:** Explicit action/reasoning phases work well with tool calls
- **Enforcing systematic approach:** When you need guaranteed output structure

Configuration:

```
Python
from google.adk.planners import PlanReActPlanner

agent = LlmAgent(
    model="your-model", # Any model, especially non-Gemini
    planner=PlanReActPlanner()
)
```

Example output structure:

According to ADK docs, PlanReActPlanner produces structured output like:

```
None
[user]: Help me plan a team offsite

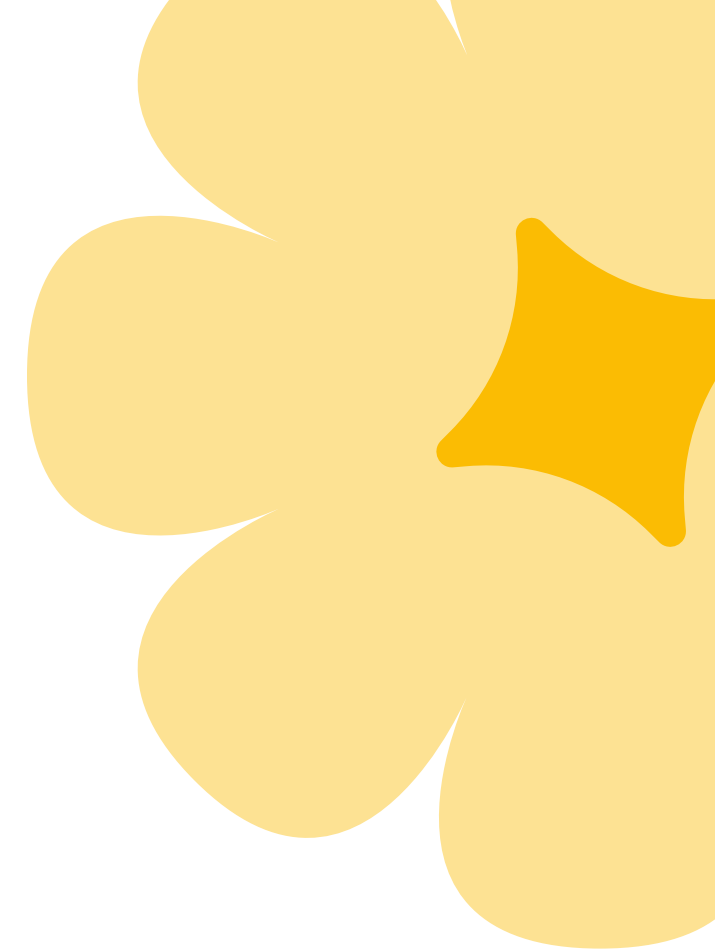
[agent]: /*PLANNING*/
1. Determine team size and budget constraints
2. Suggest venue options based on team size
3. Propose activity schedule
4. Provide logistics checklist

/*ACTION*/
[Any tool calls would appear here]

/*REASONING*/
A successful team offsite needs both structured activities and free time.
Mixing team-building exercises with casual social time creates better
engagement than all-business or all-social approaches.

/*FINAL_ANSWER*/
Here's a recommended plan for your team offsite:

Venue: Consider [specific recommendations]
Schedule: Morning workshops, afternoon team activities, evening social time
Budget: $150-200 per person for venue + activities
...
```

Which planner should you use?

Use **BuiltInPlanner** when working with Gemini models (2.5 Flash, 2.5 Pro, 2.0 Flash)

- More natural reasoning process
- Configurable thinking depth with `thinking_budget`
- Optional visibility into reasoning with `include_thoughts`

Use **PlanReActPlanner** when:

- Working with non-Gemini models that lack built-in thinking
- You need strict output structure (PLANNING/ACTION/REASONING/FINAL_ANSWER)
- Building tool-heavy agents where explicit action phases help

For this course, we focus on **BuiltInPlanner** since we're using Gemini models. **PlanReActPlanner** is covered in advanced multi-agent courses.

4. When to use planning

Use planning for:

- ✓ Multi-step problem solving
- ✓ Tasks requiring trade-off analysis
- ✓ Complex decision making
- ✓ Debugging agent reasoning

Skip planning for:

- ✗ Simple, direct questions
- ✗ Quick factual lookups
- ✗ Single-step tasks
- ✗ Time-sensitive responses

Python

```
# Good use of planning
strategic_agent = LlmAgent(
    model="gemini-2.5-flash",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(
            include_thoughts=True,
            thinking_budget=2048 # Large budget for complex analysis
        )
    ),
    instruction="Analyze business strategies and provide recommendations"
)

# Planning unnecessary
simple_agent = LlmAgent(
    model="gemini-2.5-flash",
    # No planner - simple task
    instruction="Greet users warmly"
)
```


5. Debugging with visible thoughts

Enable `include_thoughts=True` to see the agent's reasoning:

```
Python
debug_agent = LlmAgent(
    model="gemini-2.5-flash",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(
            include_thoughts=True, # See internal reasoning
            thinking_budget=1024
        )
    ),
    instruction="Help users debug problems systematically"
)

# Response will show:
# 1. The agent's internal thinking process
# 2. How it broke down the problem
# 3. The final answer
```

Hands-on example

Let's build a problem-solving agent that uses planning to tackle complex questions.

Step 1: Create the project

```
Shell
adk create problem_solver
cd problem_solver
```



Step 2: Write the planning-enabled agent

Replace the contents of `agent.py` with:

```
Python
"""
Problem-solving agent with built-in planning capabilities.
Demonstrates ADK's BuiltInPlanner with ThinkingConfig.
"""

from google.adk.agents import LlmAgent
from google.adk.planners import BuiltInPlanner
from google.genai import types

# Planning-enabled agent for complex problem solving
root_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="strategic_problem_solver",
    description="Solves complex problems using multi-step reasoning and planning",
    instruction="""You are a Strategic Problem Solver.

Your approach to complex problems:
1. **Understand** - Break down the problem into components
2. **Analyze** - Consider multiple approaches and trade-offs
3. **Plan** - Develop a step-by-step solution strategy
4. **Execute** - Provide clear, actionable recommendations

For complex problems:
- Think through implications and edge cases
- Consider short-term vs long-term consequences
- Identify potential risks and mitigation strategies
- Provide reasoning for your recommendations

Be thorough, analytical, and systematic in your approach.""",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(
            include_thoughts=True,    # Show reasoning process
            thinking_budget=2048      # Large budget for complex thinking
        )
    )
)
```


Step 3: Run and test

Shell
`adk web`

Test 1: Complex multi-step problem

Try this prompt:

None
`How should I prepare for a career transition from software engineering to product management over the next 12 months?`

Expected behavior with planning:

- Agent shows its thinking process
- Breaks down the transition into phases and skill requirements
- Considers multiple preparation strategies
- Analyzes what skills to develop and in what order
- Provides a structured, phased roadmap

None
`[Thinking Process - visible because include_thoughts=True]
- Analyzing skill gap: engineering skills (have) vs PM skills (need to develop)
- PM core competencies: product strategy, user research, stakeholder management, roadmapping
- Transferable skills from SWE: technical knowledge, data analysis, project execution
- Timeline: 12 months allows for gradual transition while employed
- Strategy: build skills → gain PM experience → networking → transition

[Final Answer]
Here's a 12-month roadmap for your transition from software engineering to product management:

Months 1-3: Build foundational PM knowledge
- Read key PM books (Inspired, Lean Product Playbook)
- Take online PM course (Reforge, Product School)
- Start writing product specs for your current team's features
- Shadow your current PM in meetings`

Months 4-6: Develop core PM skills

- Lead a small feature end-to-end (from spec to launch)
- Conduct user interviews for your team's product
- Practice stakeholder management in current role
- Build portfolio of product artifacts (specs, roadmaps, metrics analysis)

Months 7-9: Gain PM experience and visibility

- Volunteer for PM-adjacent projects (roadmap planning, user research)
- Start PM side project to demonstrate skills
- Network with PMs (informational interviews, PM meetups)
- Get PM mentorship from someone in the role

Months 10-12: Active job search

- Update resume highlighting PM-relevant achievements
- Apply to APM programs or PM roles at smaller companies
- Leverage internal opportunities if available
- Be prepared to take IC PM role (not senior) initially

This gradual approach lets you build skills while employed and reduces risk.

Test 2: Simple direct question (notice the difference)

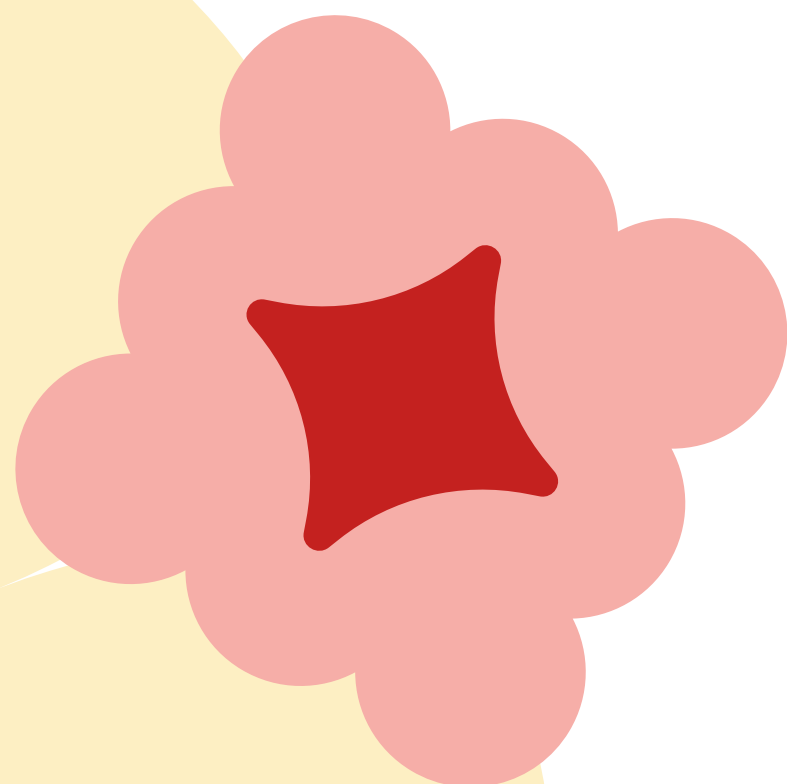
Try this prompt:

None

What is the capital of France?

Expected behavior:

- Even with planning enabled, simple questions get direct answers
- Minimal thinking overhead for straightforward queries
- The agent adapts complexity to the task



Test 3: Trade-off analysis

Try this prompt:

None

Should I rent or buy a home in San Francisco if I plan to stay for 3–5 years?

Expected behavior:

- Thinks through both options systematically
- Considers financial implications (down payment, monthly costs, equity, opportunity cost)
- Analyzes market-specific factors (SF housing prices, rent control, appreciation)
- Evaluates the 3–5 year timeline impact
- Provides context-aware recommendation with scenarios

What to notice

- Thinking process is visible when `include_thoughts=True`
- Complex problems get structured analysis with planning
- Simple questions remain efficient even with planner enabled
- Budget limits thinking depth (2048 tokens = thorough analysis)

Key takeaways

- `planner` parameter (singular) enables multi-step reasoning
- `BuiltInPlanner` leverages Gemini's built-in thinking capabilities
- `ThinkingConfig` controls thinking budget and thought visibility
- `include_thoughts=True` shows internal reasoning (great for debugging)
- `thinking_budget` allocates tokens for thinking (separate from response)
- Use planning for complex tasks, skip for simple queries
- Import from `google.adk.planners` and `google.genai` types